

UNIT II– LM 5

CLOUD SERVICE MODEL

Service-Oriented Architecture (SOA)

INTRODUCTION TO SERVICE-ORIENTED ARCHITECTURE (SOA)

Service-Oriented Architecture (SOA) is a software architectural approach in which business functionalities are designed as **independent, reusable services** that communicate with one another through **standard communication protocols** over a network.

A service is a self-contained software component that performs a specific business function, such as customer management, payment processing, inventory control, or order management. These services can be reused by multiple applications without modifying their internal implementation.

Unlike traditional monolithic applications, where all modules are tightly integrated into a single system, SOA divides the application into independent services. Each service performs a specific task and interacts with other services through standardized interfaces. This makes the system more flexible, scalable, maintainable, and interoperable.

SOA enables organizations to integrate applications developed using different programming languages, operating systems, and databases. Since services communicate using open standards such as HTTP, XML, SOAP, REST, and JSON, different applications can exchange information seamlessly regardless of the underlying technology.

SOA is widely adopted in enterprise computing environments because it allows organizations to reuse existing business services, reduce software development costs, improve productivity, and rapidly respond to changing business requirements.

Features of SOA

- Services are **loosely coupled**, allowing independent development and maintenance.
- Services communicate through **standard interfaces and protocols**.
- Business functionalities are exposed as reusable services.
- Services are platform-independent and language-independent.
- SOA promotes interoperability among heterogeneous applications.
- Services can be discovered and reused by different business applications.
- New business applications can be created by combining multiple existing services.
- Supports scalability and flexibility in enterprise systems.
- Improves software maintainability and reduces development time.

Working of SOA

In Service-Oriented Architecture, a client application sends a request for a particular business service. The service provider receives the request, processes it, accesses the required database if necessary, and returns the response to the client. Multiple applications can reuse the same service without rewriting the business logic.

Examples of business services include:

- Customer Service
- Payment Service
- Order Service
- Inventory Service
- Authentication Service
- Shipping Service

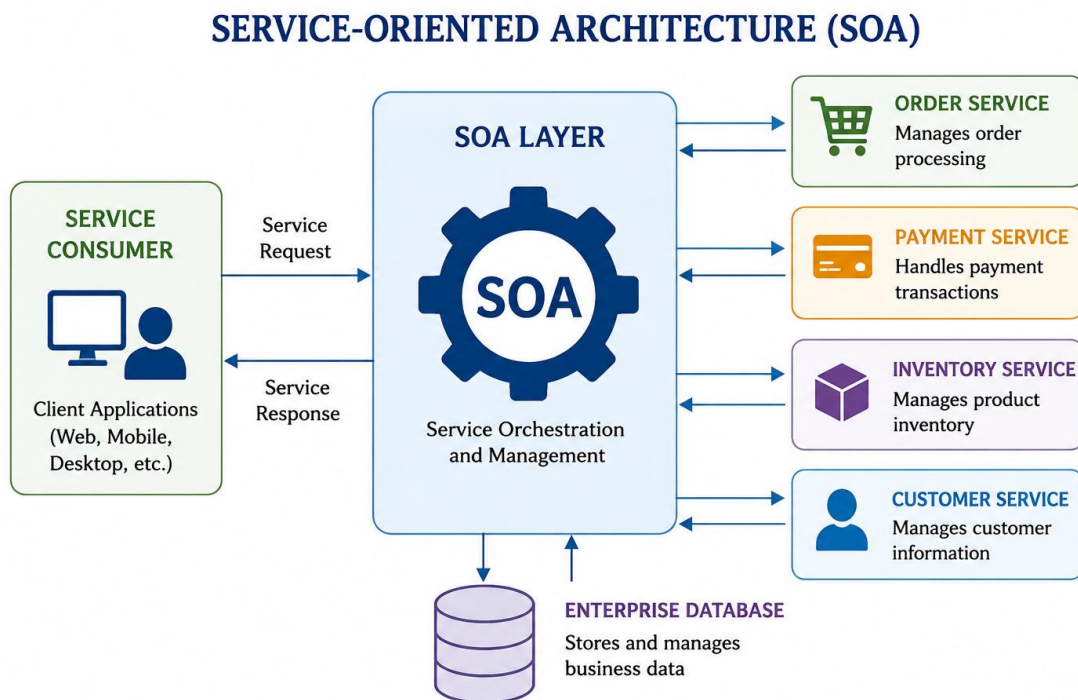


Fig. 1.1 Service-Oriented Architecture (SOA)

The Service Consumer represents the client application requesting a business function. The SOA layer receives the request and forwards it to the appropriate service. Each service performs a specific business operation independently. The required data is retrieved from the enterprise database, and the processed result is returned to the client.

Since every service is independent, the same service can be reused by multiple applications without duplication of business logic.

Key Points

- SOA stands for **Service-Oriented Architecture**.
- Business functionality is exposed as reusable services.
- Services are loosely coupled and platform independent.
- SOA improves interoperability and scalability.
- Communication takes place through standard protocols such as SOAP and REST.
- Multiple applications can reuse the same business service.
- SOA is widely used in enterprise applications such as banking, healthcare, e-commerce, ERP, and cloud computing.

1.2 NEED FOR SERVICE-ORIENTED ARCHITECTURE (SOA)

Modern organizations use multiple software applications to perform various business activities such as customer management, inventory control, payment processing, human resource management, and supply chain management. Most of these applications are developed at different times using different programming languages, operating systems, and database technologies. As a result, integrating these heterogeneous applications becomes difficult, expensive, and time-consuming.

Traditional monolithic applications are tightly coupled, making it difficult to modify, maintain, or scale individual business functions. Whenever a business requirement changes, the entire application often needs to be modified and redeployed. This increases development cost and reduces organizational flexibility.

Service-Oriented Architecture (SOA) addresses these challenges by exposing business functionalities as reusable and loosely coupled services. Each service performs a specific business task and can be accessed independently by different applications through standardized communication protocols.

Why Do We Need SOA?

- Integrates heterogeneous applications developed using different technologies.
- Promotes software reuse by allowing the same service to be used by multiple applications.
- Reduces software development and maintenance costs.
- Supports rapid application development by reusing existing services.
- Improves interoperability between different enterprise systems.
- Enables organizations to quickly respond to changing business requirements.
- Increases scalability by allowing independent deployment of services.
- Simplifies maintenance because changes in one service do not affect others.
- Facilitates cloud computing and distributed application development.

- Improves overall business agility and operational efficiency.

Real-Time Example

A banking organization offers Internet Banking, Mobile Banking, ATM, and Branch Banking systems. Instead of developing separate customer verification modules for each application, a single **Customer Authentication Service** is created. All banking applications reuse this service, reducing development effort and ensuring consistency.

1.2 NEED FOR SOA

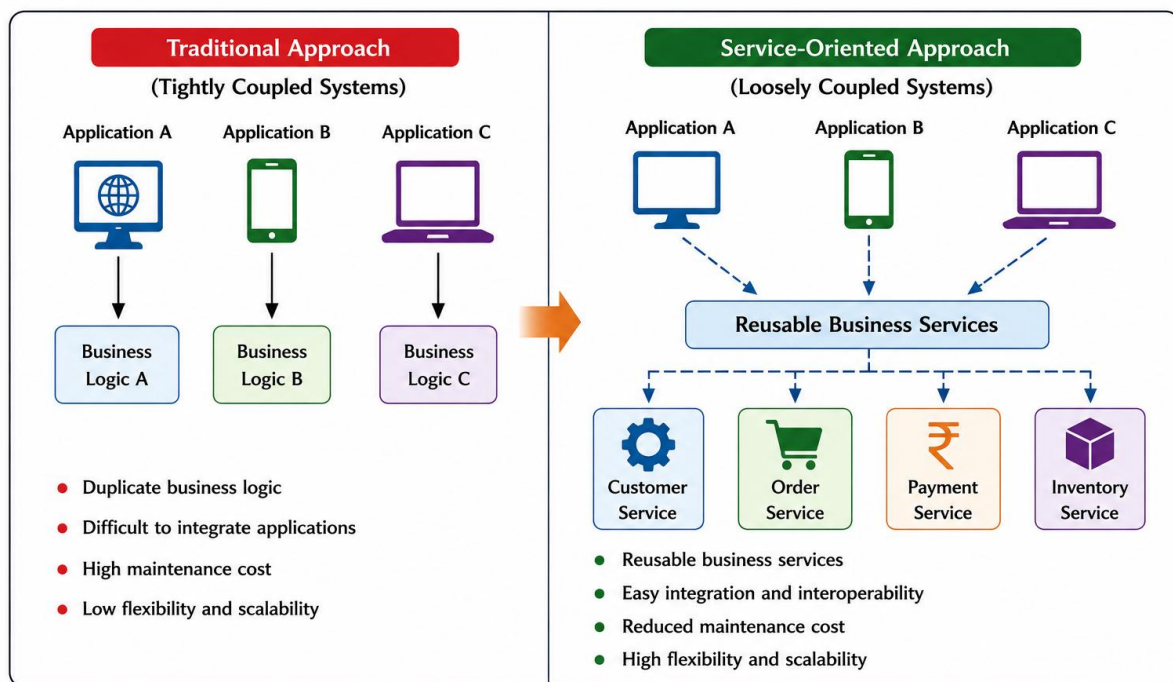


Fig. 1.2 Need for SOA

CHARACTERISTICS OF SERVICE-ORIENTED ARCHITECTURE (SOA)

The following characteristics make SOA one of the most popular enterprise software architectures.

1. Loose Coupling

Each service is independent of other services. A change made to one service does not affect the functioning of other services. This improves flexibility and simplifies maintenance.

2. Reusability

A service developed once can be reused by multiple applications without rewriting the same business logic.

3. Interoperability

Services developed using different programming languages and operating systems can communicate through standard protocols such as SOAP, REST, XML, and JSON.

4. Discoverability

Services are published in a service registry, allowing developers and applications to discover and use them whenever required.

5. Composability

Multiple services can be combined to create a new business application or composite service.

6. Autonomy

Each service independently manages its own business logic, resources, and execution environment.

7. Scalability

Individual services can be scaled independently based on workload without affecting the entire application.

8. Platform Independence

Services communicate through open standards, making them independent of programming language, operating system, or hardware platform.

Table 1.1 Characteristics of SOA

Characteristic	Description
Loose Coupling	Services are independent of each other.
Reusability	Same service can be reused in different applications.
Interoperability	Services communicate across different platforms.
Discoverability	Services can be located through a registry.
Composability	Multiple services can be combined.

Autonomy	Services control their own execution.
Scalability	Individual services can be scaled independently.
Platform Independence	Services work across heterogeneous environments.

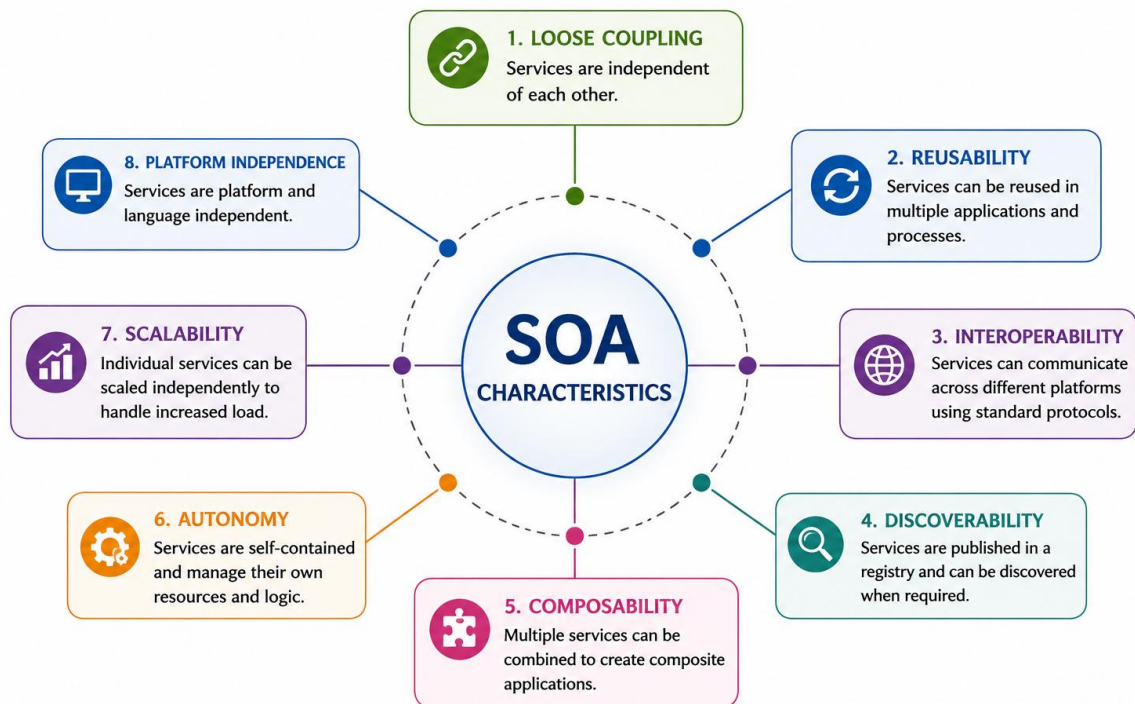


Fig. 1.3 Characteristics of SOA

DESIGN PRINCIPLES OF SOA

Design principles provide guidelines for developing efficient, reusable, and maintainable services.

1. Standardized Service Contract

Each service should expose a well-defined interface describing the operations, inputs, outputs, and communication protocol.

2. Loose Coupling

Services should minimize dependencies on one another.

3. Service Reusability

Services should be designed in a generic manner so they can be reused across multiple business applications.

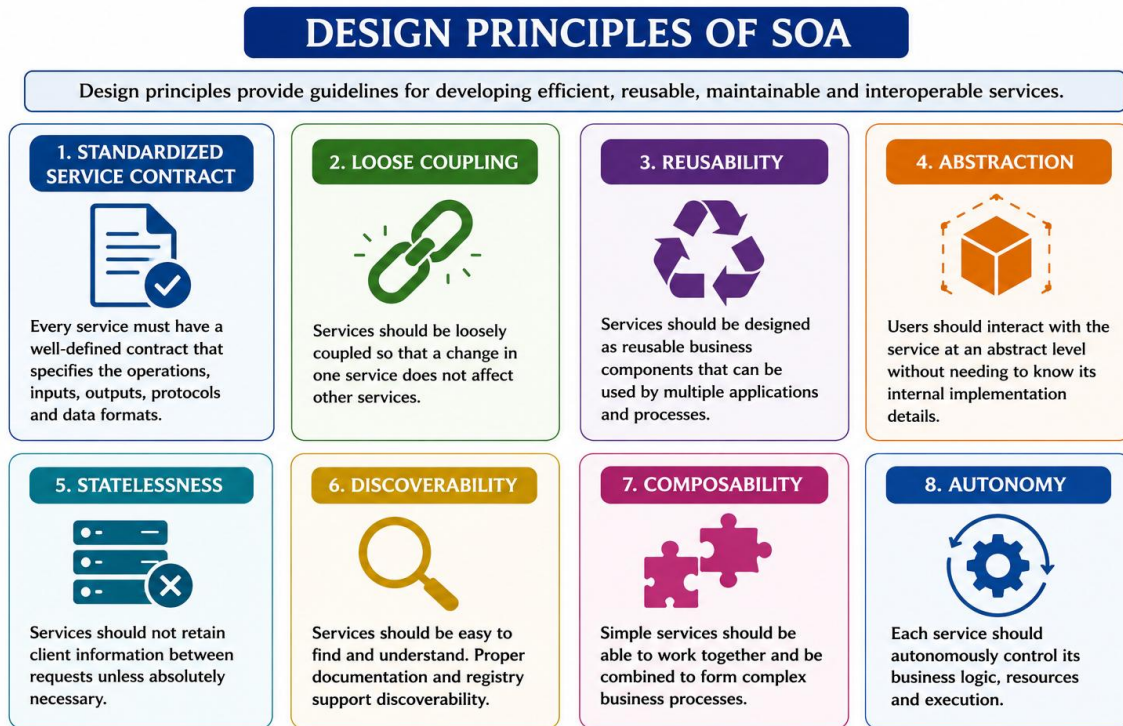


Fig. 1.4 Design Principles of SOA

4. Service Abstraction

Users should only see the service interface. Internal implementation details should remain hidden.

5. Service Autonomy

Each service independently manages its execution and business logic.

6. Statelessness

Services should avoid storing unnecessary client information between requests.

7. Discoverability

Services should be easy to identify through documentation or service registry.

8. Composability

Simple services should be capable of being combined into more complex business processes.

Table 1.2 Design Principles of SOA

Principle	Description
Standardized Service Contract	Common interface specification
Loose Coupling	Independent services
Reusability	Service used multiple times
Abstraction	Hide implementation details
Statelessness	No unnecessary session information
Discoverability	Easy to locate services
Composability	Build larger applications from services
Autonomy	Independent execution

COMPONENTS OF SERVICE-ORIENTED ARCHITECTURE (SOA)

SOA consists of several components that work together to deliver business services efficiently.

1. Service Provider

Develops, hosts, and publishes business services for consumers.

2. Service Consumer

Application or user requesting the required business service.

3. Service Registry

A repository where services are registered and discovered.

4. Service Contract

Defines service interface, operations, input/output parameters, policies, and communication protocols.

5. Enterprise Service Bus (ESB)

Acts as middleware to route messages, transform data formats, provide security, and coordinate communication among services.

6. Business Services

Actual reusable business functionalities such as Order Service, Payment Service, Inventory Service, Customer Service, and Shipping Service.

Table 1.3 Components of SOA

Component	Function
Service Provider	Publishes services
Service Consumer	Uses services
Service Registry	Stores service information
Service Contract	Defines service interface
Enterprise Service Bus	Routes and manages communication
Business Services	Perform business operations

Common Use Cases for SoA

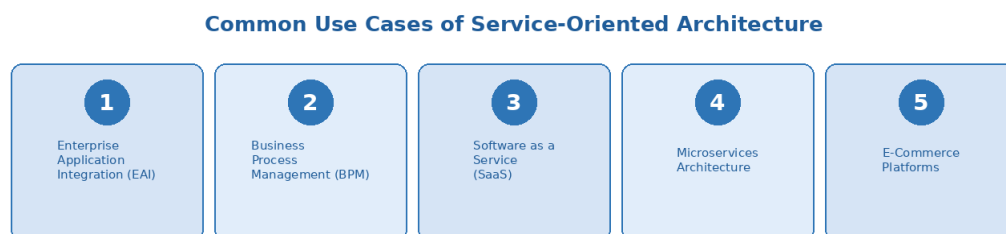


Fig. Common Use Cases for SoA

Challenges of SoA



Fig.Challenges of SoA

Popular Technologies for Implementing SoA

1. Web Services

- **SOAP (Simple Object Access Protocol):** A protocol for exchanging structured information in web services.
- **REST (Representational State Transfer):** An architectural style for designing networked applications using standard HTTP methods.
- **WSDL (Web Services Description Language):** An XML-based language for describing web service interfaces, operations, and endpoints.
- **UDDI:** Universal Description, Discovery, and Integration — a registry standard for publishing and discovering web services.

2. Enterprise Service Bus (ESB)

- Middleware that provides the necessary infrastructure for service communication, routing, and transformation (e.g., MuleSoft, Apache Camel, IBM Integration Bus).

3. API Management Tools

- Tools to create, manage, and secure APIs (e.g., Apigee, AWS API Gateway, Azure API Management, Kong).

4. Messaging Systems

- Systems for asynchronous communication between services (e.g., Apache Kafka, RabbitMQ, ActiveMQ).

5. Service Discovery Tools

- Tools for registering and discovering services (e.g., Consul, Eureka, etcd).